



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster en Inteligencia Artificial

Optimización de modelos de lenguaje compactos para la síntesis de programas orientada a la resolución de ARC-AGI-1

Trabajo Fin de Estudios

presentado por: Asier Marin Fernandez

Dirigido por: Fernando Antonio Rufo Jimenez

Ciudad: Valencia

Fecha: 22 de julio de 2026

Índice de Contenidos

Resumen	VII
Abstract	VIII
1. Introducción	1
1.1. Contexto	1
1.2. Justificación	2
1.3. Objetivos	2
2. Estado del Arte	4
2.1. Definición de la inteligencia	4
2.2. La inteligencia desde un punto de vista filosófico	4
2.3. Núcleos del conocimiento	5
2.4. Sistemas de pensamiento	6
2.5. Algoritmos de búsqueda y optimización	6
2.5.1. Búsqueda en anchura	8
2.5.2. Búsqueda de coste uniforme	8
2.5.3. Búsqueda en profundidad	9
2.5.4. Búsqueda A*	10
2.5.5. Algoritmo de Monte Carlo aplicado a Búsquedas	10
2.5.6. Síntesis de programas	13
2.5.7. Búsqueda Guiada por Redes Neuronales	13
2.6. Abstract and Reasoning Corpus	14
2.6.1. Espectro de la generalización	15
2.6.2. La inteligencia desde la perspectiva de la adquisición de habilidades	16
2.6.3. ARC-AGI-1	19
2.6.4. Núcleos de conocimiento aplicado en ARC-AGI	20
2.6.5. Repositorio de datos	23
2.6.6. ARC-AGI-2	24
2.6.7. ARC-AGI-3	24
2.6.8. Comparativa entre ARC-AGI-1, ARC-AGI-2 y ARC-AGI-3	24
2.7. Primeros enfoques para resolver ARC-AGI	25

2.7.1. Domain Specific Languages (DSLs) y síntesis de programas	25
2.7.2. Enfoque Neurosimbólico	25
2.7.3. LARC	25
2.8. Grandes Modelos de Lenguaje en ARC-AGI-1	25
2.9. Modelos basados en la Neurodiversidad	26
2.9.1. Inducción de Síntesis de programas mediante Redes Neuronales Efi- cientes	26
2.10. Modelos basados en Transformers compactos	26
2.11. Enfoques de modelos sin Pre-entrenamiento	26
2.12. Optimización en Tiempo de Inferencia	26
2.13. Combinación de sistemas de programas con modelos de transducción	26
2.14. Modelos compactos basados en bucles de refinamiento iterativo	26
2.14.1. Modelos de razonamientos jerárquicos	26
2.14.2. Modelos de razonamientos recursivos con redes neuronales compactas	27
2.15. Estado actual con perspectiva de eficiencia computacional	27
3. Metodología	28
3.1. Enfoque Científico y Método de Investigación	28
3.2. Metodología de Desarrollo y Gestión del Proyecto	28
3.3. Ciclo de Vida del Modelo basado en MLOps	29
4. Infraestructura y herramientas	30
4.1. Infraestructura de computación	30
4.2. Entorno de desarrollo	31
4.3. Utilidades externas de ARC-AGI	33
5. Desarrollo	34
5.0.1. Técnicas de aumento de datos	34
5.1. Arquitectura y entrenamiento	34
6. Resultados	35
7. Planificación	36
8. Estudio económico	37

9. Conclusiones y Trabajo Futuro	38
Referencias	38
A. Apéndices	40

Índice de Ilustraciones

2.1. Representación árbol de nodos búsqueda en anchura	8
2.2. Representación árbol de nodos búsqueda en profundidad	9
2.3. Pasos algoritmo MCTS	12
2.4. Modelo jerárquico de habilidades cognitivas y su mapeo del espectro de generalización	17
2.5. Posición del problema, un sistema inteligente que genera programas de ha- bilidades	18
2.6. Tarea donde el objetivo es completar un patrón de una area dada	21
2.7. Prueba de ruido	22
2.8. El objetivo rojo debe “moverse” junto al objeto azul hasta hacer “contacto”	22
2.9. Tarea donde el objetivo es contar el objeto que aparece más veces	23
2.10. Tarea donde el objetivo es generar un línea diagonal hasta rebotar al hacer contacto con el obstáculo rojo	23

Índice de Tablas

4.1. Rendimiento por nivel de precisión y formato	31
4.2. Librerías de entorno en Python, versión y abreviación	32

Resumen

Nota: En este apartado se introducirá un breve resumen en español del trabajo realizado (extensión máxima: 150 palabras). Este resumen debe incluir el objetivo o propósito de la investigación, la metodología, los resultados y las conclusiones.

Palabras Clave: Se deben incluir de 3 a 5 palabras claves en español

Abstract

Nota: En este apartado se introducirá un breve resumen en español del trabajo realizado (extensión máxima: 150 palabras). Este resumen debe incluir el objetivo o propósito de la investigación, la metodología, los resultados y las conclusiones.

Palabras Clave: Se deben incluir de 3 a 5 palabras claves en inglés

1. Introducción

Actualmente una tendencia ascendente en el campo de inteligencia artificial. El paradigma del aprendizaje profundo ha demostrado ser una herramienta de presente y de futuro. El *software* es un elemento fundamente en la sociedad actual, y con la evolución del propio *software* también evolucionan los distintos campos tecnológicos, entre ellos el de la inteligencia artificial.

La inteligencia artificial es un campo de estudio que se centra en la creación de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, el razonamiento, la resolución de problemas, la comprensión del lenguaje natural y la percepción visual. En los últimos años, ha habido un aumento significativo en el interés y el desarrollo de la inteligencia artificial, lo que ha llevado a avances notables en áreas como el aprendizaje automático, el procesamiento del lenguaje natural y la visión por computadora.

Cada vez se le dá más importancia a las herramientas basadas en inteligencia artificial y cada vez llega a un público más amplio. En este contexto, los mecanismos para que esta tecnología pueda ser consumida por el público general se vuelven cada vez más importantes. En este sentido, la eficiencia en el consumo de estos recursos tecnológicos cada vez es más relevante. Con la llegada de más modelos de lenguaje, la necesidad de encontrar formas de medir la capacidad de generalización de estos modelos se vuelve cada vez más importante también.

Es aquí donde entra en juego el *Abstraction and Reasoning Corpus (ARC-AGI)*, un *benchmark* diseñado específicamente para evaluar la inteligencia de los sistemas y su capacidad para adquirir nuevas habilidades de forma eficiente a partir de unos pocos ejemplos, basándose en los principios del conocimiento central innato humano. El estándar ARC-AGI es una respuesta a la necesidad de medir la inteligencia, permitiendo la comparación entre sistemas y humanos.

1.1. Contexto

A pesar de los avances recientes, los Grandes Modelos de Lenguaje siguen teniendo dificultades para resolver este tipo de tareas, ya que sus capacidades se derivan principalmente de volúmenes masivos de datos de pre-entrenamiento y memorización probabilística.

Esto evidencia que, la hipótesis de escalar los modelos sin límites insospechados, no es suficiente para alcanzar el razonamiento abstracto. Aumentar ciegamente la cantidad de parámetros no dota automáticamente al sistema la capacidad de resolver tareas que requieren un razonamiento abstracto más novedoso.

ARC-AGI representa el indicador estándar por excelencia de la industria para medir el progreso real hacia la inteligencia general artificial a través de la generalización amplia y síntesis de programas.

1.2. Justificación

Abordar el problema de ARC-AGI desde la perspectiva de los Modelos de Lenguaje Pequeños o SMLs y la optimización en tiempo de inferencia supone un cambio de paradigma necesario: pasar del escalado por fuerza bruta a la optimización inteligente de los parámetros. Limitar deliberadamente la capacidad del modelo evita la sobre-generalización y fuerza al sistema a concentrarse exclusivamente en descubrir transformaciones algorítmicas robustas y reglas recursivas, en lugar de memorizar estadísticas superficiales.

La viabilidad de este enfoque está respaldada por investigaciones muy recientes. Arquitecturas como el *Hierarchical Reasoning Model* y el *Tiny Recursive Model* han logrado superar el rendimiento de modelos masivos en ARC-AGI utilizando redes minúsculas de apenas 7 a 27 millones de parámetros, gracias a la aplicación de pasos de computación recursiva en un espacio latente. Asimismo, enfoques revolucionarios como CompressARC han demostrado que es posible resolver un alto porcentaje de puzzles sin absolutamente ningún pre-entrenamiento, utilizando apenas 76.000 parámetros y minimizando la longitud de descripción puramente en el tiempo de inferencia (*Test-Time Training*).

Dadas las restricciones de capacidad de cómputo disponibles para este proyecto, apostar por estos paradigmas (bucles de refinamiento, síntesis de programas y entrenamiento en tiempo de prueba) no solo es una necesidad técnica, sino la estrategia científica más prometedora y eficiente para abordar el razonamiento complejo sin la saturación de recursos inherente a los modelos de lenguaje a gran escala.

1.3. Objetivos

Objetivo principal El objetivo principal es diseñar y desarrollar un modelo compacto (SLM) que maximice la puntuación en la resolución del benchmark ARC-AGI-1, priori-

zando estrictamente la eficiencia computacional y el rendimiento por parámetro frente a los modelos de lenguaje a gran escala.

En cuanto a objetivos particulares se enumeran los siguientes:

- Desarrollar e implementar bucles de refinamiento recursivo que permitan al modelo actualizar de manera iterativa sus estados latentes y mejorar progresivamente sus predicciones, simulando la síntesis de programas.
- Explorar y aplicar técnicas de adaptación en tiempo de prueba *Test-Time Training* y *Test-Time Compute* para que el modelo aprenda y descubra abstracciones algorítmicas directamente sobre las tareas objetivo, reduciendo la dependencia del pre-entrenamiento.
- Integrar mecanismos de tiempo de cálculo adaptativo o *early-stopping* que permitan al sistema abortar el bucle de razonamiento iterativo una vez haya alcanzado una solución convergente, optimizando drásticamente el consumo de memoria y tiempo de ejecución.
- Implementar estrategias de consistencia y ensamblado *top-k* que operen sobre diversas variantes o aumentos de los datos de entrada para seleccionar las predicciones más robustas.
- Realizar una comparativa exhaustiva de eficiencia que evalúe cualitativamente la relación entre la precisión lograda en el benchmark y el coste computacional empleado, como el uso de VRAM, y precisión en FLOPs, validando la viabilidad de la inteligencia algorítmica en entornos de recursos restringidos.

2. Estado del Arte

2.1. Definición de la inteligencia

La definición de inteligencia no es una tarea sencilla. A lo largo de la historia, filósofos, científicos y expertos en diversas disciplinas han propuesto diferentes definiciones de inteligencia, cada una con su propia perspectiva y enfoque. A pesar de muchos intentos, no existe una definición estándar de inteligencia.

La inteligencia se ha descrito de manera aproximada pero no se ha llegado a una definición concreta. Según la real academia española, la inteligencia es la “capacidad de entender o comprender”, “capacidad de resolver problemas” y “conocimiento, comprensión, acto de entender”. Sin embargo, esta definición es bastante amplia y no captura completamente la complejidad de lo que implica la inteligencia.

Para crear una definición única se debe tener en cuenta varios factores. La inteligencia es una propiedad que tiene en consideración la interacción con uno o diversos entornos. La inteligencia también se relaciona con la capacidad del agente para tener éxito u obtener ganancias al respecto de alguna meta. De la misma forma, también dependen de cómo el agente está capacitado en diferentes objetivos y entornos. En base a referencias de diversos autores, en este trabajo se adoptará el estándar de definición más utilizada a día de hoy.

“Intelligence measures an agent’s ability to achieve goals in a wide range of environments.”

(Legg, Hutter, y cols., 2007, p. 9, párr. 6)

2.2. La inteligencia desde un punto de vista filosófico

Autores como Descartes, Kant o Hume han reflexionado sobre la naturaleza de la inteligencia y la mente humana. Descartes, por ejemplo, propuso la idea de que la mente y el cuerpo son entidades separadas, lo que se conoce como dualismo (González, 2011). Kant, por su parte, argumentó que la inteligencia es una facultad innata que nos permite organizar y comprender el mundo a través de categorías y conceptos (Waxman, 2014). Hume, en cambio, se centró en la experiencia y la percepción como base de la inteligencia humana.

Aristóteles también realiza varias reflexiones sobre lo que hoy se entiende por inteligencia. En concreto, sus reflexiones se desarrollan fundamentalmente a través de la naturaleza del intelecto, la facultad de inteligir y el pensamiento.

“El intelecto -siendo impasible- ha de ser capaz de recibir la forma, es decir, ha de ser en potencia tal como la forma pero sin ser ella misma y será respecto de lo inteligible algo análogo a lo que es la facultad sensitiva respecto de lo sensible.”

(Aristotle y Boeri, 2010, p. 230, párr. 1)

2.3. Núcleos del conocimiento

En la teoría se ha considerado que el conocimiento humano se basa en un único sistema de aprendizaje de propósito general. Investigaciones recientes del departamento de Psicología de la Universidad de Harvard ha propuesto una teoría alternativa, que sugiere el conocimiento humano se basa en una serie de núcleos o módulos innatos de conocimiento innatos, cada uno especializado en un dominio específico, como el espacio, los objetos, las personas, los números y el lenguaje (Spelke y Kinzler, 2007).

Cada sistema se define por un conjunto de principios y un conjunto de fronteras cognitivas. Estas fronteras actúan como un sello de identidad que ayuda a diversos investigadores a demostrar que existe un sistema cognitivo único (Spelke y Kinzler, 2007). Por ejemplo, el sistema numérico central, se evidencia con representaciones numéricas poco imprecisas, esta imprecisión crece de forma lineal a medida que aumenta el número representado.

1. **Sistema de representación de objetos inanimados:** Es la capacidad de percibir límites de objetos, la forma completa o parcial de objetos ocultos y percibir cuándo y hacia donde se moverán los objetos.
2. **Sistema de representación de agentes y sus acciones:** Es la capacidad de percibir agentes animados, sus acciones y sus intenciones, incluso sin lenguaje.
3. **Sistema de representación numérica:** Es la capacidad de representar cantidades numéricas, incluso sin lenguaje, y de realizar operaciones aritméticas básicas.
4. **Sistema de representación geométrica del espacio:** Es la capacidad de representar el espacio y las relaciones espaciales entre objetos.

5. **Sistema de representación del lenguaje y social:** Es la capacidad de comprender y utilizar el lenguaje, así como de interactuar socialmente con otros individuos.

2.4. Sistemas de pensamiento

El concepto de sistemas de pensamiento fue originalmente propuesto por los psicólogos Keith Stanovich y Richard West, quienes describieron dos sistemas de pensamiento: el sistema 1, que es rápido, automático e intuitivo, y el sistema 2, que es lento, deliberado y analítico. Posteriormente fue popularizado por el ganador del Nobel Daniel Kahneman en su libro *Thinking, Fast and Slow* (2011).

- **Sistema 1 (Pensamiento intuitivo o rápido):** Este es el sistema de pensamiento más rápido, automático e intuitivo. Funciona de manera automática y es responsable de nuestras «reacciones instintivas», juicios rápidos y habilidades aprendidas que se han vuelto algo natural, como reconocer la cara de un amigo o resolver $2+2$. En el contexto de la inteligencia artificial, esto es análogo a la transducción, donde un modelo realiza directamente una predicción global basada en patrones.
- **Sistema 2 (Pensamiento analítico o lento):** Este es nuestro modo de pensamiento lento, deliberado y lógico. Requiere atención consciente y se utiliza para tareas complejas, como resolver un problema matemático difícil, sopesar ventajas e inconvenientes o aprender una nueva habilidad. En el ámbito de la inteligencia artificial, esto se corresponde con la inducción o la síntesis de programas, donde un sistema construye deliberadamente un programa lógico paso a paso para llegar a una solución.

La clave de estos conceptos es que el pensamiento humano hace uso de una combinación de ambos sistemas, tiene la capacidad de usar un pensamiento más transductor o inductivo dependiendo de la tarea y situación. Para la mayoría de tareas diarias es más eficiente el uso de un pensamiento más transductor, pero cuando se requiere un razonamiento más complejo o intensivo, el pensamiento inductivo es más adecuado.

2.5. Algoritmos de búsqueda y optimización

Teóricamente, casi cualquier problema puede ser resuelto por fuerza bruta, pero en la realidad el coste computacional y tiempo hace inabarcable este enfoque. El problema

y el espacio son características fundamentales para determinar la estrategia de búsqueda y optimización más adecuada. En este sentido, existen diferentes algoritmos de búsqueda y optimización que se pueden aplicar en función de las características del problema y el espacio.

Tradicionalmente los algoritmos de búsqueda se han clasificado en dos grandes categorías, los algoritmos de búsqueda no informada y los algoritmos de búsqueda informada. Los algoritmos de búsqueda no informada, son aquellos algoritmos que no están guiados o simplemente están basados en fuerza bruta. No existe información sobre el dominio del problema, sólo representación. Estos algoritmos intentan explorar todas las opciones posibles en cada nodo. Para buscar una solución óptima se imponen una serie de restricciones o reglas con el objetivo de evitar caer en bucles o ciclos de la estructura de los datos. Algunos ejemplos de algoritmos de búsqueda no informada son; búsqueda en anchura (BFS), búsqueda en profundidad (DFS), búsqueda de coste uniforme, búsqueda iterativa o limitada en profundidad, entre otros.

Los algoritmos no informados pueden ser mejorados proporcionando más información sobre el problema. Búsqueda informada significa que el algoritmo tiene un contexto específico y la heurística ayuda a representar el contexto. La heurística puede ser una serie de reglas para evaluar un estado y puede medir su rendimiento. De esta manera, no existe un camino fijo y los nodos son visitados de acuerdo al coste heurística. Los costes son calculados a medida que se recorre el árbol de cada nodo visitado y se añade a la cola. El objetivo es simple, ignorar los árboles cuyo coste es mayor que los visitados, ahorrando coste computacional. La heurística se puede definir como la capacidad de realizar innovaciones para conseguir alcanzar los objetivos.

“Un proceso que puede resolver un problema dado, pero que no ofrece ninguna garantía de que lo hará, se llama una heurística para ese problema.”

(Newell, Shaw, y Simon, 1962)

La heurística puede reducir drásticamente el coste computacional de problemas que son resueltos con algoritmos no informados, pero también puede llevar a soluciones sub-óptimas. Algunos algoritmos de búsqueda informados son; búsqueda el primero el mejor (BFA), búsqueda codiciosa el primero el mejor (GBFS), búsqueda A*, búsqueda de haz (Beam Search), entre otros.

2.5.1. Búsqueda en anchura

Explora todas las opciones de un determinado nivel de profundidad antes de pasar al siguiente nivel. Es necesario visitar todos los nodos hijos de un determinado nivel y finalizar al llegar al objetivo. Para implementarlo se usa una cola FIFO ó *First In First Out*, los nodos de un nivel son procesados y sus nodos hijos son añadidos a la cola. Puede ser óptimo si todas las acciones tienen el mismo coste y es completo siempre que el espacio sea discreto o finito.

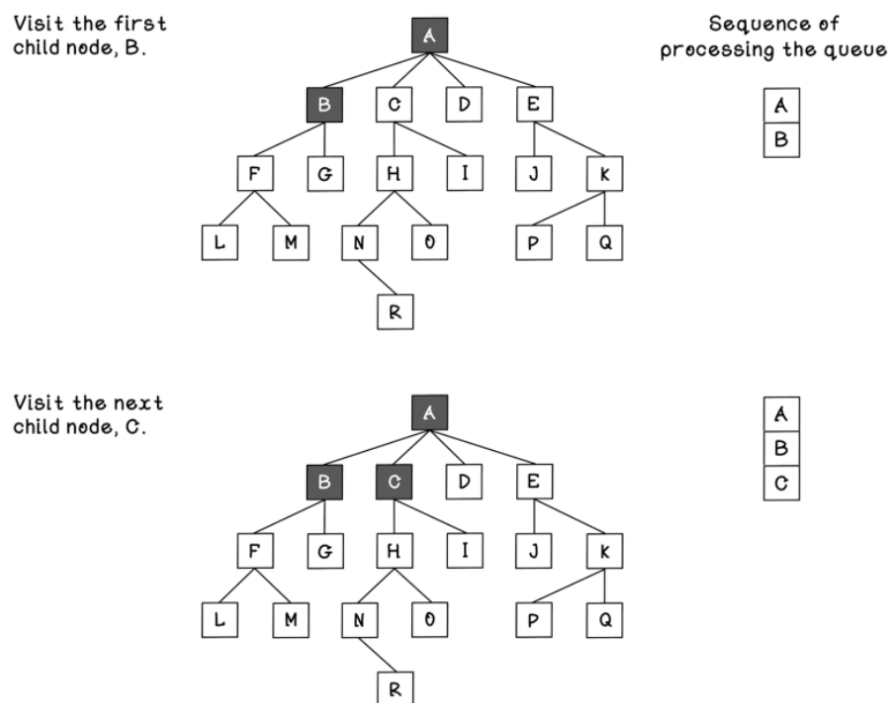


Figura 2.1: Representación árbol de nodos búsqueda en anchura.

Fuente: Adaptado de “Grokking Artificial Intelligence Algorithms: Understand and apply the core algorithms of deep learning and artificial intelligence” (Hurbans, 2020)

2.5.2. Búsqueda de coste uniforme

Se trata de una variación de la búsqueda en anchura, pero en este caso se tiene en cuenta el coste de cada acción. El nodo con el menor coste acumulado es expandido primero. Para implementarlo se usa una cola de prioridad, donde los nodos con menor coste acumulado tienen mayor prioridad. Es óptimo siempre que el coste de las acciones sea no negativo y es completo siempre que el espacio sea discreto o finito.

En caso de que el coste sea el mismo en todos los nodos, la búsqueda de coste uniforme se comporta exactamente igual que la búsqueda en anchura, ya que el orden de expansión de los nodos será el mismo. Sin embargo, si los costes varían, la búsqueda de coste uniforme puede ser más eficiente al expandir primero los nodos con menor coste acumulado, lo que puede llevar a encontrar una solución óptima más rápidamente que la búsqueda en anchura.

2.5.3. Búsqueda en profundidad

Explora un nodo hijo antes de explorar los nodos hermanos. Repite el proceso para el resto de hijos que ya ha visitado. Para implementarlo se usa una pila LIFO ó *Last In First Out*, el nodo más recientemente visitado es el primero en ser procesado.

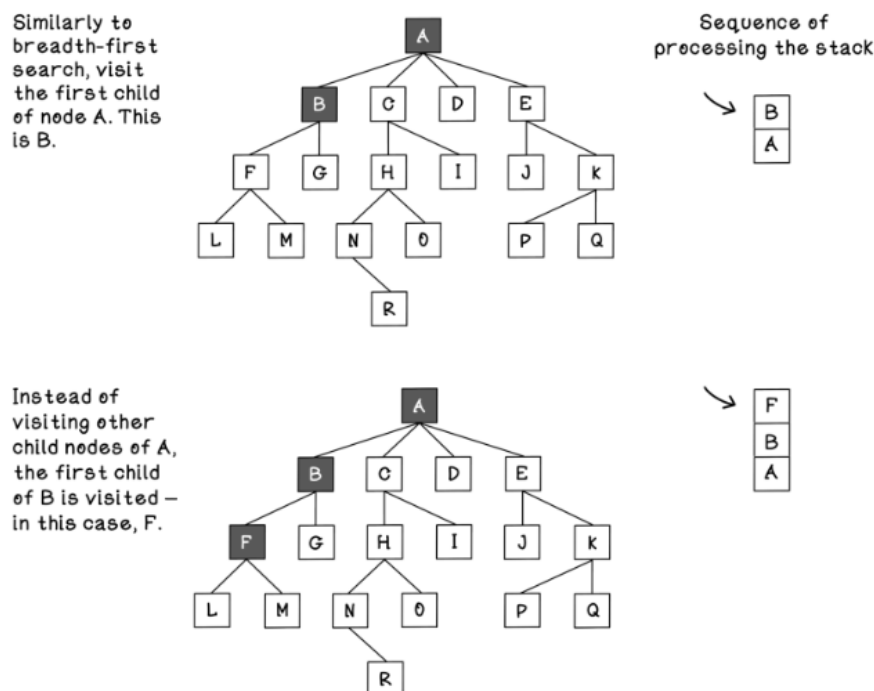


Figura 2.2: Representación árbol de nodos búsqueda en profundidad.

Fuente: Adaptado de “Grokking Artificial Intelligence Algorithms: Understand and apply the core algorithms of deep learning and artificial intelligence” (Hurbans, 2020)

No es completo en caso de entorno no finito. Tampoco es óptimo en caso de entorno no finito. Es aconsejable utilizar la búsqueda en profundidad de manera limitada, se considera que los nodos por debajo del límite no tienen hijos.

También existe la alternativa al uso del algoritmo en profundidad iterativa, incrementando el límite del nivel de profundidad de manera iterativa, hasta que se alcance el

objetivo. Es completo y óptimo siempre que el coste de las acciones sea el mismo.

2.5.4. Búsqueda A*

Existen distintas variantes de algoritmos de búsquedas informadas *Best Search*. La mayoría de estos algoritmos emplean una función heurística estimar el coste estimado de la ruta a nivel computacional más eficiente. Por ejemplo, el algoritmo el primero el mejor codicioso (GBFS) es el caso más básico, se expande aquel nodo que parece más cerca del objetivo según la estimación heurística.

La búsqueda A* es el más utilizado de esta categoría. Se considera el coste tanto el coste acumulado desde el nodo raíz, como el coste estimado mediante heurística hasta el objetivo. Es completo siempre que la heurística sea admisible. También se puede considerar óptimo y una complejidad temporal y espacial de $O(b^d)$, donde b es el factor de ramificación y d es la profundidad de la solución óptima. Para que la heurística sea admisible, nunca debe sobrestimar el coste real y debe ser consistente. Se asume que el coste de cualquier transición es mayor que 0.

La combinación de búsquedas y la satisfacción de restricciones es una técnica habitual. La idea de CSP es representar problemas mediante la declaración de restricciones sobre un dominio y encontrar soluciones que satisfagan esas restricciones.

2.5.5. Algoritmo de Monte Carlo aplicado a Búsquedas

En diversos problemas o juegos, se puede construir, al menos en teoría, un espacio que se asemeja a un árbol de estados completo que puede contener todos los resultados posibles. Un algoritmo como Minimax se basa precisamente en la idea de evaluar la bondad de las jugadas de etiquetas de estados que se propagan hasta un estado actual. El objetivo es decidir cual es el camino o árbol que conduce a mejores resultados al jugador. El problema que se encuentra con el algoritmo Minimax y derivados es que el espacio de búsqueda puede ser tan grande que resulta absolutamente inabarcable. En todos los algoritmos se han proporcionado métodos para mitigar el problema mediante acotaciones de la profundidad de búsqueda hacia adelante ó directamente la poda de ramificaciones para evitar visitar aquellos nodos que parecen resultar menos prometedores.

Como respuesta a este tipo de aproximaciones hace unos años surgió la técnica de *Monte Carlo Tree Search* (Búsqueda en Árboles con Monte Carlo) que ofrece algunas ventajas inherentes. Se puede aplicar en espacios de búsqueda con muchas ramificaciones, espacios

que antes eran inabarcables. No requiere conocimiento específico del juego y no siempre tiene porqué encontrar una solución óptima. *Upper Confidence Bound applied to Trees* (UCBT) es la variante más común que trata de equilibrar la exploración y explotación de nodos. La idea general es simular jugadas al azar desde una posición actual para realizar una muestra de cómo se comportan las distintas. El algoritmo del límite superior de confianza (UCB) trata de equilibrar la exploración y explotación de nodos. Para que UCBT no genere un gran número de simulaciones aleatorias y minimizar riesgos, la variante UCT hace uso de estadísticas de bondad disponibles para todos los estados sucesores. MCTS cuenta con diversas fases:

- **Selección:** Se parte del nodo raíz y en cada nivel se elige la rama más prometedora utilizando una función de utilidad como UCT. Se continúa descendiendo hasta alcanzar un nodo terminal o un nodo que aún tiene acciones sin explorar.
- **Expansión:** Si el nodo seleccionado no está completamente explorado, se elige una de sus acciones no visitadas. Se crea un nuevo nodo hijo y se añade al árbol. Se crea un nuevo nodo hijo y se añade al árbol.
- **Simulación:** Desde el nuevo nodo se simula una partida o secuencia completa de acciones hasta llegar a un estado final. La simulación puede ser aleatoria o guiada por heurísticas. El resultado proporciona una recompensa o valoración de la rama explorada.
- **Retro-propagación:** La recompensa obtenida se propaga hacia atrás por todos los nodos recorridos durante la iteración. Se actualizan sus estadísticas y esta información se utilizará en futuras selecciones para decidir qué ramas explorar.

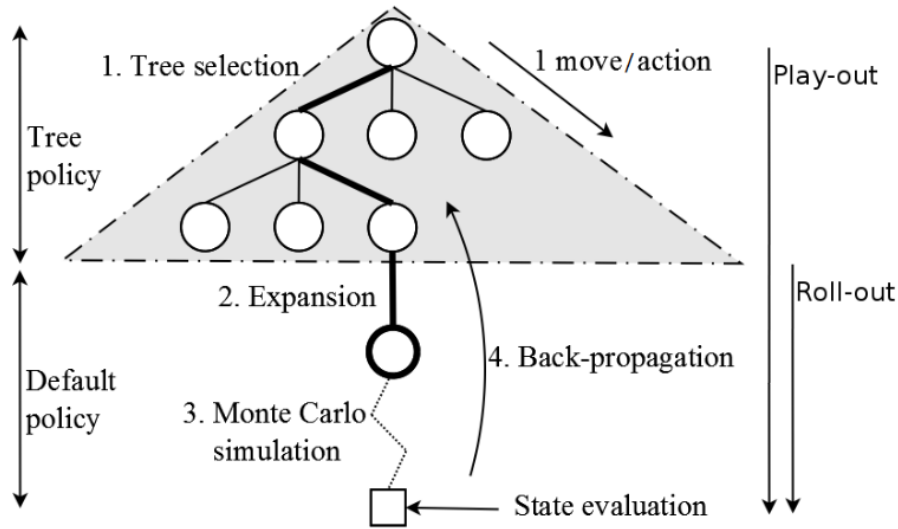


Figura 2.3: Pasos del algoritmo MCTS.

Fuente: Adaptado de “Knowledge-based fast evolutionary MCTS for general video game playing” (Perez, Samothrakis, y Lucas, 2014)

MCTS selecciona la rama más prometedora, expande una acción nueva, simula el resultado y actualiza las estadísticas del camino recorrido para mejorar futuras decisiones. Se puede definir tres estrategias fundamentales que intervienen en distintas fases del algoritmo. Durante la fase de selección es necesario decidir qué nodo del árbol debe explorarse a continuación. Esta decisión puede tomarse utilizando diferentes criterios, como elegir el nodo con mayor recompensa acumulada, el que haya participado en un mayor número de simulaciones exitosas o, más habitualmente, aplicar una función que equilibre exploración y explotación, como UCB. En la fase de expansión se determina qué nuevo nodo añadir al árbol (Perez y cols., 2014). Aunque lo más común es seleccionar una acción disponible de forma aleatoria, también puede aprovecharse conocimiento específico del problema para guiar esta elección. Finalmente, en la fase de simulación se genera una secuencia completa de acciones desde el nuevo nodo hasta alcanzar un estado final. Estas simulaciones pueden realizarse de forma aleatoria o incorporar heurísticas y conocimiento del dominio para obtener estimaciones más precisas.

Actualmente existen variantes como Adaptive Branching MCTS (AB-MCTS) que introduce un mecanismo de ramificación adaptativa para mejorar la eficiencia en espacios de búsqueda con alta ramificación, o Sakana AI, un sistema de inteligencia artificial que utiliza MCTS para resolver puzzles del benchmark ARC-AGI.

2.5.6. Síntesis de programas

La síntesis de programas es un área de investigación que se centra en la generación automática de programas a partir de especificaciones o ejemplos. El objetivo es crear programas que cumplan con ciertos requisitos o resuelvan problemas específicos sin necesidad de escribir código manualmente.

Existen diferentes enfoques para la síntesis de programas, como la síntesis basada en ejemplos, donde se proporcionan ejemplos de entrada y salida para que el sistema genere un programa que cumpla con esos ejemplos. Otro enfoque es la síntesis basada en especificaciones, donde se proporcionan especificaciones formales del comportamiento deseado y el sistema genera un programa que satisfaga dichas especificaciones.

2.5.7. Búsqueda Guiada por Redes Neuronales

Históricamente, los algoritmos de búsqueda clásicos como A* ó MCTS, funcionaban explorando sistemáticamente las posibles acciones futuras. Sin embargo, en dominios complejos como el ajedrez, o la síntesis de programas para ARC-AGI, el número de ramificaciones posibles crece de forma exponencial en cada paso (Świechowski, Godlewski, Sawicki, y Mańdziuk, 2023), generando una explosión combinatoria que hace imposible la resolución por fuerza bruta dentro de un límite de tiempo o memoria manejable.

Por otro lado, las Redes Neuronales Profundas son excelentes reconociendo patrones e intuiciones de un solo vistazo, pero carecen de la capacidad algorítmica para planificar deliberadamente a largo plazo o corregir sus propios errores secuenciales. La Búsqueda Guiada por Redes Neuronales soluciona las carencias de ambos mundos inyectando la "intuición" del aprendizaje profundo dentro de la estructura lógica de los árboles de búsqueda. Esto se materializó entrenando redes neuronales para que actuaran como entes evaluadoras dentro de algoritmos como MCTS.

En su versión clásica, MCTS depende de simulaciones masivas para estimar la calidad de cada rama. Sin embargo, en los enfoques híbridos modernos se incorporan redes neuronales que guían de forma explícita la exploración:

- **Red de Política (*Policy Network*):** En lugar de que el árbol de búsqueda explore todas las acciones legales posibles por igual, la red neuronal predice una distribución de probabilidad sobre cuáles son las acciones más prometedoras. Esto reduce drásticamente la amplitud de la búsqueda, podando inmediatamente las ramas inútiles.

- **Red de Valor (*Value Network* / *Q-function*):** En lugar de tener que simular el juego o el programa hasta su conclusión final para saber si es exitoso, la red neuronal evalúa la calidad o “bondad” del estado intermedio actual. Esto reduce drásticamente la profundidad de la búsqueda.

Gracias a estas capacidades de generalización y abstracción, el modelo híbrido logra converger hacia la solución correcta a una velocidad de órdenes de magnitud mayor que la búsqueda clásica, haciendo tratables problemas que antes se consideraban imposibles para las máquinas.

2.6. Abstract and Reasoning Corpus

Los sistemas de evaluación de la inteligencia artificial han ido evolucionando a lo largo del tiempo. La investigación de esta área de ha centrado históricamente en comprobar si las máquinas realizan bien tareas específicas, siendo una evaluación orientada a tareas y no una evaluación de la inteligencia intrínseca.

“[AI is] the science of making machines capable of performing tasks that would require intelligence if done by [humans].”

(Minsky, 1968)

Tras los métodos tradicionales de evaluar algoritmos, la IA ha pasado a una evaluación de “caja negra” basada en el comportamiento que involucra la discriminación humana (comparación directa por o contra humanos) y pruebas de rendimiento de problemas. Se iniciaron evaluaciones metodológicas de dudosa validez, donde los investigadores a menudo seleccionaban los conjuntos de datos que mejor les convenían y ajustaban parámetros en exceso provocando sobre-ajustes. Esto llevó a situaciones donde muchos resultados publicados eran reproducibles pero difícilmente replicable (Hernández-Orallo, 2017). En consecuencia, la aparente mejora del modelo desaparecía en cuanto cambiaba ligeramente el dominio de los datos, además de una fuerte dispersión de los métodos en las diferentes disciplinas.

Para sistemas de inteligencia artificial de propósito general era necesario un nuevo enfoque orientado a habilidades. Este enfoque se basa en la evaluación de capacidades cognitivas en lugar de tareas en las que fue diseñado originalmente (Hernández-Orallo,

2017). Para garantizar la validez de este enfoque, las pruebas de evaluación requieren directrices estrictas, como definir con antelación los conjuntos de datos para entrenamiento y evaluación, y evitar el sobre-ajuste a conjuntos de datos específicos.

“The field of artificial intelligence has been very successful in developing artificial systems that perform these tasks without featuring intelligence.”

(Hernández-Orallo, 2017)

Para avanzar hacia sistemas de inteligencia artificial más inteligentes y humanos es necesario un método de evaluación de la inteligencia que compare dos o más sistemas y humanos. Es aquí donde entra en juego el Abstract and Reasoning Corpus (ARC-AGI), un *benchmark* diseñado para respetar todas las directrices necesarias. ARC-AGI está construido con un conjunto de prioridades para ser lo más parecido posible a la capacidad de razonamiento innata del ser humano (Chollet, 2019b).

Los modelos que existen en la actualidad funcionan correctamente en tareas específicas, todavía están ciertamente limitados y hambrientos de datos. En situaciones en los que se desvían ligeramente de sus datos de entrenamiento, son incapaces de afrontar tareas que requieren un razonamiento novedoso. ARC-AGI también está pensado para medir el progreso de manera precisa y cuantitativa de la inteligencia. En este sentido, las definiciones que se han dado en este trabajo en secciones anteriores puede ayudar a intuir la estructura de las pruebas de evaluación de ARC-AGI. Las tareas de evaluación no deben ser necesariamente conocidas de antemano, para conseguir la generalización el agente debe que ser capaz de aprender y gestionar nuevas tareas (Chollet, 2019b).

“We have been conceptualizing and evaluating intelligence in the context of AI research: crystallized skill on one hand, skill-acquisition ability on the other.”

(Chollet, 2019b)

2.6.1. Espectro de la generalización

La generalización es un concepto que ha llegado de la mano del aprendizaje automático, desarrollado originalmente para caracterizar el buen rendimiento de un modelo estadístico en datos de entrada que no forman parte de su conjunto de datos de entrenamiento. Este concepto ha seguido siendo parte de la conversación y se puede definir como la

”habilidad” de un modelo para gestionar una situación o varias situaciones (o tareas) que no se ha encontrado previamente. Nótese que el concepto de la frase ”situación que no se ha encontrado previamente” puede ser tremendamente ambigua y el autor (Chollet, 2019b) propone un espectro de generalización para aclarar esta ambigüedad. En este espectro, se pueden encontrar diferentes tipos de generalización, como la generalización sistémica (centrada en el sistema) o generalización consciente-desarrollada (capacidad de manejar situaciones que ni el sistema ni su creador humano habían encontrado antes) que no se va a profundizar en este artículo. Al contrario, se va a profundizar en la generalización de habilidades, que es más útil para cuantificar los grados de generalización.

- **Ausencia de generalización:** Sistemas donde no existe incertidumbre ni novedad. Entorno cerrado como un algoritmo que juega al “tres en raya”, iterando exhaustivamente sobre todas las jugadas posibles. No hay adaptación posible.
- **Generalización local o robustez:** Es la capacidad de un sistema para manejar nuevos ejemplos dentro de una distribución de datos conocida para una sola tarea o un conjunto bien definido de ellas. Este es el nivel en el que ha operado históricamente el aprendizaje profundo moderno. El autor (Chollet, 2019b) hace referencia a un ejemplo de clasificador de gatos que, se califican gatos donde el modelo no había visto antes.
- **Generalización amplia o flexibilidad:** Es la capacidad de manejar una categoría amplia de tareas y entornos relacionados sin intervención humana adicional, incluyendo situaciones imprevistas por los creadores (Chollet, 2019b). Un ejemplo de esto puede ser el coche autónomo de nivel 5.
- **Generalización extrema o generalidad:** Sistemas abiertos capaces de manejar tareas enteramente nuevas que solo comparten características abstractas con las ya conocidas, aplicable a cualquier dominio. También se puede denominar como inteligencia general.

2.6.2. La inteligencia desde la perspectiva de la adquisición de habilidades

Existen dos perspectivas fundamentales sobre la mente humana, una que es una colección de habilidades que se han grabado en nuestro ADN en la evolución y y la otra

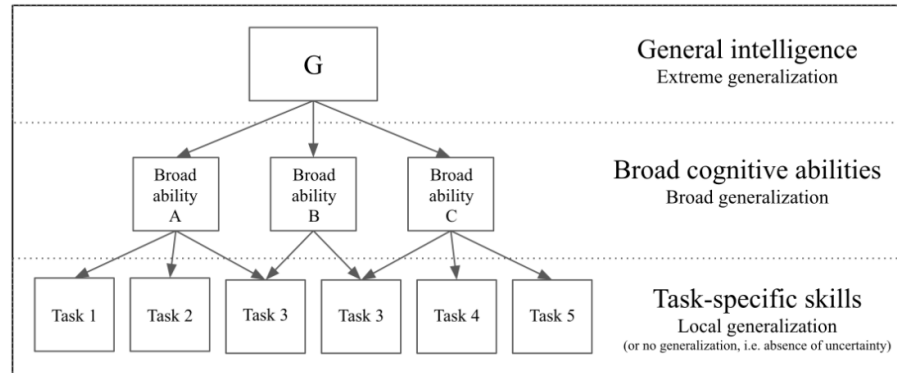


Figura 2.4: Modelo jerárquico de habilidades cognitivas y su mapeo del espectro de generalización.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

que se trata de ver la mente como una página en blanco. El autor François Chollet, en el manifiesto “On the Measure of Intelligence” (Chollet, 2019b), sugiere que los humanos no nacen con una mente vacía que aprende cualquier conocimiento desde cero., pero tampoco son una colección de instintos. Se puede decir que nacemos con algunos conocimientos previos innatos (Spelke y Kinzler, 2007).

Para que un “benchmark” como ARC pueda ser una medida real de la inteligencia no puede depender de conocimientos adquiridos y las tareas deb ser resolubles utilizando únicamente los principios de los núcleos de conocimiento. Cómo de eficiente puede ser un agente para usar usu conocimientos innatos en resolver problemas que no ha conocido antes.

François Chollet define la inteligencia no como un programa estático, sino como un proceso de síntesis de programas. Para establecer esta definición, hay que detallar en una primera instancia la posición del problema y el proceso técnico de una tarea según su marco formal. La arquitectura del sistema se compone de tres entidades principales;

- **El Sistema Inteligente (*IS*):** Es el agente que genera un programa de habilidad (*SP*) basándose la experiencia recibida.
- **La Tarea (*T*):** Es el entorno que plantea un problema a resolver. La tarea tiene la función de puntuación (éxito o fracaso) y una función de retroalimentación que devuelve al sistema inteligente.
- **El programa de habilidad (*SP*):** Artefacto producido por (*IS*), un programa

ejecutable que trata de resolver la tarea y devuelve una respuesta. Representa la capacidad del sistema para gestionar el entorno o problema dado. Un sistema es más inteligente si proporciona programas de alta habilidad usando los menos recursos posibles en tareas de alta dificultad de generalización.

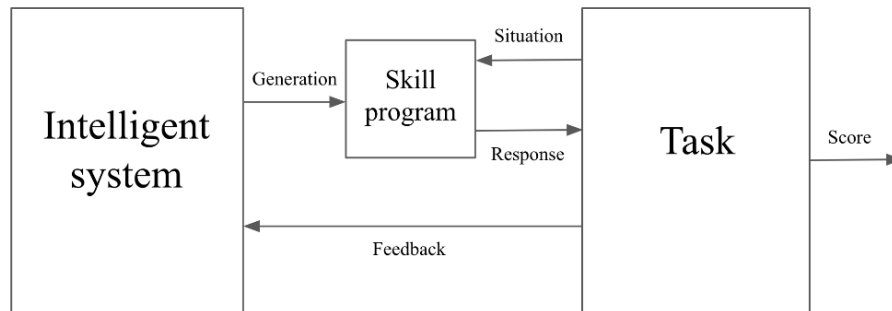


Figura 2.5: Posición del problema, un sistema inteligente que genera programas de habilidades que interactúan con una tarea.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

El proceso de la resolución de una tarea se podría dividir en dos fases:

1. **Fase de entrenamiento o adquisición de habilidades:** Ciclo interactivo donde el sistema inteligente (IS) aprende a generar un programa de habilidad (SP) a través de la experiencia con la tarea (T). En esta fase, el sistema recibe retroalimentación sobre su desempeño y ajusta su programa de habilidad en consecuencia.
 - a. Fase de generación. Se presenta una situación de entrenamiento.
 - b. Fase de Producción. El IS genera un programa de habilidad (SP).
 - c. Fase de Acción. El SP interactúa con la tarea (T) para producir una respuesta.
 - d. Fase de Retroalimentación. La tarea evalúa la respuesta, se asigna una puntuación y se genera un *feedback*.
 - e. Fase de Actualización. El sistema inteligente (IS) recibe ese feedback para actualizar su estado ó estados internos, mejorando su comprensión del problema para una siguiente iteración.
2. **Fase de evaluación o prueba de generalización:** Una vez terminada la fase de entrenamiento, el IS entrega la versión definitiva de su programa de habilidad.

En esta fase el programa se enfrenta a situaciones o entornos que no ha percibido anteriormente.

a. Fase de Desafío. Se presenta una situación nueva (test) que implica incertidumbre.

b. Fase de Ejecución. El programa de habilidad produce respuestas sin posibilidad de recibir retroalimentación.

c. Fase de Medición. Suma de la puntuación obtenida. Si el programa ha podido gestionar las situaciones nuevas significa que se ha demostrado el principio de generalización.

En conclusión, ¿qué se puede esperar de un *benchmark* de inteligencia ideal? Un *benchmark* ideal debe ser explícito sobre su alcance, se debe demostrar el éxito sobre las pruebas y que exista una correlación estadística con éxito en un entorno real. Los resultados deben ser reproducibles, tal como señalaba el autor Jose Hernandez-Orallo (2017). Además, se debería medir la capacidad de generalización, es decir, medir la capacidad de gestionar situaciones que ni el sistema ni el desarrollador ha visto anteriormente. Esto indica que el conjunto de tareas dedicadas a la evaluación debe ser estrictamente privado para evitar el se inyecte la solución o bien en el código o en los propios datos de entrenamiento (contaminación). Otro punto relevante que acompaña a este aspecto es el diseño para evitar trampas estadísticas que no requieran una verdadera comprensión abstracta.

El control de la experiencia debe ser fundamental. No se debe “comprar” el rendimiento de un *benchmark* mediante un conjunto de datos de entrenamiento ilimitados. Si un sistema inteligente necesita de millones de datos de entrenamiento para generar programas de habilidad consistentes que un humano puede captar con apenas pocos ejemplos, significa que el sistema es ineficiente y no inteligente aun que logre la misma puntuación. La cantidad de datos de entrenamiento sería estrictamente limitada. Por último, el *benchmark* debe ser funcional de la misma forma para humanos y máquinas, de forma justa, asumiendo que ambos agentes poseen los mismos núcleos de rendimiento y que requiere tiempo y tamaño del entrenamiento de un humano.

2.6.3. ARC-AGI-1

Con el objetivo de dar respuesta a todas las necesidades que se han mencionado en las secciones anteriores, nace la propuesta de *benchmark* Abstraction and Reasoning

Corpus (ARC), un repositorio de datos con la intención de servir como referencia para medir la inteligencia general. En esta sección se va a explorar los objetivos y el diseño de este repositorio.

ARC-AGI como *benchmark* de síntesis de programas tiene diferentes objetivos finales:

- Mantener un formato cercano o simple para que pueda ser solucionable por humanos.
- Centrar el foco en medir la generalización y no sólo habilidades. La generalización debe medir de una manera cuantitativa. Presentar tareas altamente abstractas que deben ser entendidas por el ser humano con pocos ejemplos.
- Control cuantitativo de la experiencia proporcionada (datos de entrenamiento limitados). Los datos de entrenamiento tendrán una cantidad fija.
- Describir el conjunto completo de núcleos de conocimiento y comparación “precisa” con la inteligencia entre el ser humano y los agentes de inteligencia artificial.

En la imagen siguiente 2.6 se muestra un ejemplo de una tarea de ARC-AGI-1. En esta tarea, el objetivo es completar un patrón de una area dada. La solución de esta tarea pasa por generar una cuadrícula de salida que corresponde con la entrada a su izquierda. Para resolver esta tarea, el agente debe ser capaz de identificar el patrón subyacente en la entrada y generar la salida correcta, demostrando así su capacidad para generalizar a partir de los ejemplos proporcionados.

2.6.4. Núcleos de conocimiento aplicado en ARC-AGI

Todos los problemas planteados están diseñados específicamente para abordar los núcleos de conocimiento. Los nucleos de conocimiento de ARC están planteados para ser lo más parecido posible a los núcleos de conocimiento del ser humano.

Objetualidad

Núcleo de conocimiento centrado en la capacidad de interpretación del entorno, analizando las diferentes entidades. Se fundamenta bajo el principio cohesión y persistencia de objetos. Se analiza y se separa la cuadrícula de la entrada en diferentes objetos basándose en valores o criterios de continuidad como los bloques del mismo color.

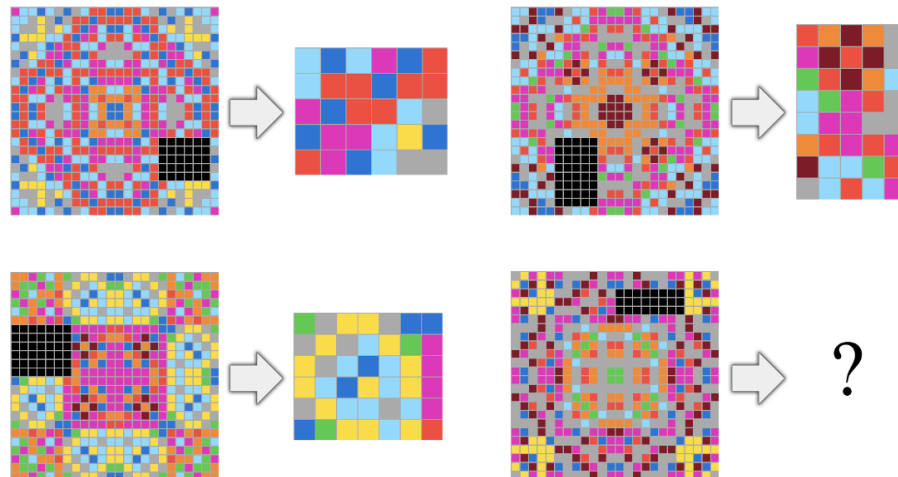


Figura 2.6: Tarea donde el objetivo es completar un patrón de una area dada. Para solucionar esta tarea debe generar una cuadrícula de salida que corresponde con la entrada a su izquierda.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

Generalmente, los objetos presentes en la cuadrícula de entrada persisten en la salida, a menudo transformando algún transformación geométrica. A pesar del ruido visual o la inclusión provocada por otros objetos, muchos de los objetos persisten.

La influencia mediante el contacto es otro punto especialmente relevante, muchas tareas requieren de entender ciertas interacciones físicas, como un objeto que se desplaza hasta tocar a otro o una línea que se dibuja y rebota al entrar en contacto con un obstáculo.

Orientación a objetivos

Las cuadrículas de entrada y salida suelen ser comprendidas por los humanos como los estados iniciales y finales de un proceso que conlleva intencionalidad. Las transformaciones buscan alcanzar una meta y resulta ser una heurística útil para resolver puzzles.

Números y conteo

Diversas tareas de ARC implican contar y clasificar objetos. Las tareas pueden tener distintos tamaños y es especialmente relevante la comparación de magnitudes. No es sólo contar objetos o “puntos”, es también identificar aquellos símbolos que aparecen en más ocasiones (conurrencia). Es necesario nociones básicas de suma y resta en cantidades pequeñas. En ARC las cantidades utilizadas no tienen un número mayor de 9. Esto se refleja

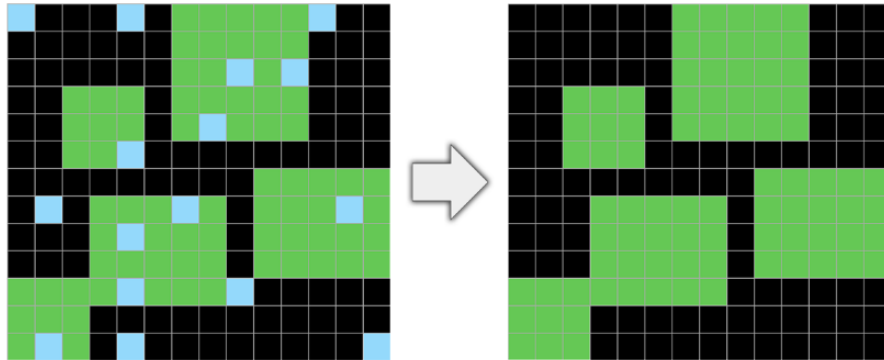


Figura 2.7: Prueba de ruido (*denoising*).

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

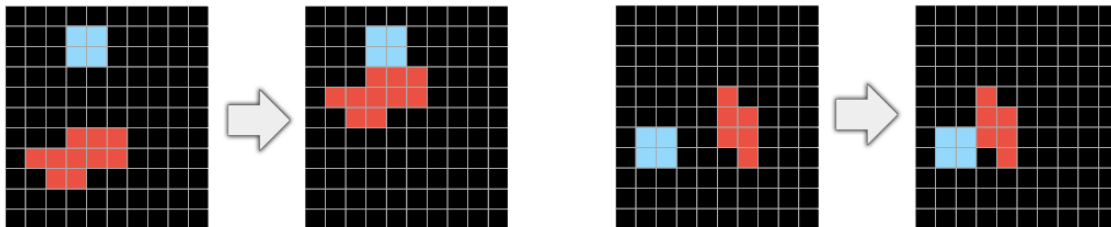


Figura 2.8: El objetivo rojo debe “moverse” junto al objeto azul hasta hacer “contacto”.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

como una capacidad innata de los humanos para la representación numérica, en secciones anteriores se ha explorado esta capacidad como uno de los núcleos de conocimiento innato. El ser humano es capaz de usar esta capacidad sin ninguna formación previa con limitación en la cantidad de objetos que contar.

Geometría y Topología básica

La orientación espacial es un aspecto fundamental como núcleo de conocimiento. Se engloban aquellas tareas de reconocimiento de formas para la compresión de líneas u otras figuras geométricas. El agente debe ser capaz de estimar con mayor probabilidad la aparición de formas regulares frente a otras formas más complejas.

Las transformaciones espaciales como las simetrías y rotaciones, así como el escalado, deben ser comprendidas y aplicables. Relaciones espaciales específicamente en nociones topológicas sobre contener o ser contenido y estar fuera o dentro de un perímetro delimitado también será requerido en diversas tareas. El trazado de líneas o la conexión de puntos (proyecciones ortogonales) o la copia de diversos objetos repetidamente también se incluyen

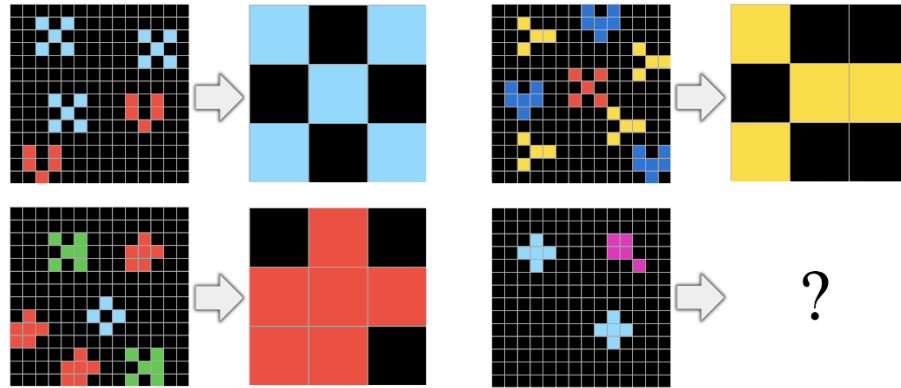


Figura 2.9: Tarea donde el objetivo es contar el objeto que aparece más veces.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

en este núcleo de conocimiento.

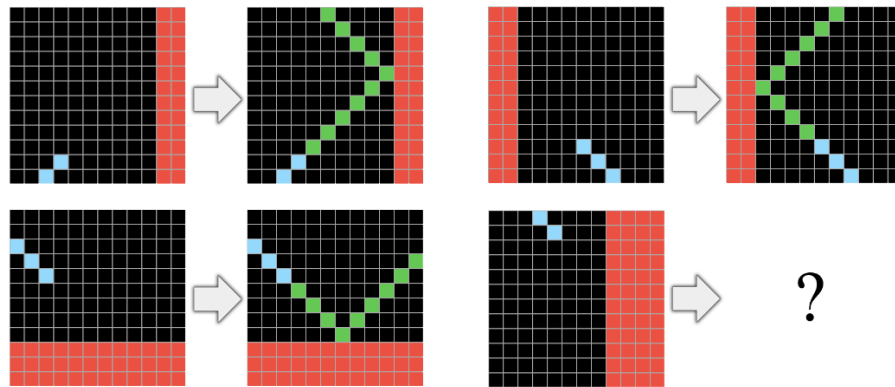


Figura 2.10: Tarea donde el objetivo es generar una línea diagonal hasta rebotar al hacer contacto con el obstáculo rojo.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

2.6.5. Repositorio de datos

Los datos de ARC-AGI se pueden encontrar en el repositorio de GitHub (Chollet, 2019a). Los datos están contenidos en dos sub-directorios diferentes:

- **training:** Contiene 400 tareas de entrenamiento. Cada tarea contiene todo el contenido en un archivo, por lo tanto, son 400 archivos. Todos los archivos de este directorio sirven para el entrenamiento del modelo y la adquisición de habilidades.
 - a. Pares “train”: Típicamente suele tener tres o más pares de “arrays”. Cada par de “arrays” contiene un “array” de “input” y otro de “output”.

b. Pares “test”: Típicamente suele tener un par de “arrays”. El par de “arrays” contiene un “array” de “input” y otro de “output”.

- **evaluation:** Contiene 400 tareas de evaluación. Cada tarea contiene todo el contenido en un archivo, por lo tanto, son otros 400 archivos.

Los archivos están en formato JSON. Cada archivo contiene un diccionario con dos claves: “train” y “test”. El valor de cada clave es una lista de pares de “arrays”. Cada par de “arrays” contiene un “array” de “input” y otro de “output”. Los “arrays” son matrices bidimensionales de números enteros que representan los estados iniciales y finales de una tarea.

En las siguientes matrices se puede ver una matriz de input X y una matriz de output Y :

$$X = \begin{bmatrix} 0 & 7 & 7 \\ 7 & 7 & 7 \\ 0 & 7 & 7 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 0 & 0 & 7 & 7 & 7 & 7 & 7 & 7 \\ 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 7 & 7 & 0 & 7 & 7 & 0 & 7 & 7 \\ 7 & 7 & 7 & 7 & 7 & 7 & 7 & 7 & 7 \\ 0 & 7 & 7 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 0 & 0 & 7 & 7 & 7 & 7 & 7 & 7 \\ 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \end{bmatrix}$$

2.6.6. ARC-AGI-2

Investigado. Pendiente de redactar.

2.6.7. ARC-AGI-3

Investigado. Pendiente de redactar.

2.6.8. Comparativa entre ARC-AGI-1, ARC-AGI-2 y ARC-AGI-3

Investigado. Pendiente de redactar.

2.7. Primeros enfoques para resolver ARC-AGI

2.7.1. Domain Specific Languages (DSLs) y síntesis de programas

En 2020 Johan Sokrates Wind publicó uno de los primeros intentos basado en un Lenguaje de Dominio Específico ó DSL. El uso de código estrictamente diseñado para resolver tareas específicas de ARC-AGI permitió alcanzar una puntuación de 20 % en el benchmark. La idea detrás de esto es codificar algoritmos de programación para crear un conjunto de programas primitivos capaz de modificar matrices bidimensionales.

Los algoritmos realizan búsquedas exhaustivas sobre el espacio de programas posibles que se puede construir cambiando núcleos de conocimiento. Esta mecánica se puede considerar casi fuerza bruta. Formalmente, el método evalúa de manera iterativa funciones unarias sobre fragmentos de las cuadrículas de entrada, formando un Grafo Acíclico Dirigido (DAG) que representa múltiples secuencias de transformaciones.

Según el propio autor, la búsqueda de nivel 4, siendo de las más altas, dura entorno a 9 horas usando solamente CPU. Este enfoque no requiere de ningún pre-entrenamiento y sus conocimientos previos son inyectados mediante líneas de código. El espacio de búsqueda crece de manera exponencial a medida que aumenta el tamaño del DSL y para los puzzles que requieren de más pasos lógicos, la búsqueda simplemente sin tiempo. Otro aspecto negativo del enfoque es que si un problema de transformación no se programó específicamente, el sistema simplemente no es capaz de resolverlo. Escalar este enfoque significa que es necesario de capacidad humana constante para seguir añadiendo conocimientos primitivos y heurística de poda diseñada manualmente para guiar la búsqueda (Wind, 2020).

2.7.2. Enfoque Neurosimbólico

Investigado. Pendiente de redactar.

2.7.3. LARC

Investigado. Pendiente de redactar.

2.8. Grandes Modelos de Lenguaje en ARC-AGI-1

Investigado. Pendiente de redactar.

2.9. Modelos basados en la Neurodiversidad

Investigado. Pendiente de redactar.

2.9.1. Inducción de Síntesis de programas mediante Redes Neuronales Eficientes

Investigado. Pendiente de redactar.

2.10. Modelos basados en Transformers compactos

Investigado. Pendiente de redactar.

2.11. Enfoques de modelos sin Pre-entrenamiento

Investigado. Pendiente de redactar.

2.12. Optimización en Tiempo de Inferencia

Investigado. Pendiente de redactar.

2.13. Combinación de sistemas de programas con modelos de transducción

Investigado. Pendiente de redactar.

2.14. Modelos compactos basados en bucles de refinamiento iterativo

Investigado. Pendiente de redactar.

2.14.1. Modelos de razonamientos jerárquicos

Investigado. Pendiente de redactar.

2.14.2. Modelos de razonamientos recursivos con redes neuronales compactas

Investigado. Pendiente de redactar.

2.15. Estado actual con perspectiva de eficiencia computacional

Investigado. Pendiente de redactar.

3. Metodología

3.1. Enfoque Científico y Método de Investigación

El presente trabajo se enmarca dentro del método científico empírico-analítico. Dado que el objetivo principal es evaluar la eficiencia y capacidad de un modelo de lenguaje compacto en la resolución del benchmark ARC-AGI-1, el enfoque adoptado es de carácter cuantitativo. La investigación sigue un proceso hipotético-deductivo articulado en etapas: planteamiento del problema ante los límites de la hipótesis de escala en modelos LLM, formulación de la hipótesis basada en modelos compactos, diseño experimental, ejecución y análisis métrico de los resultados.

Desde la perspectiva investigadora, se aplica un método empírico experimental. Este método es el más idóneo ya que permite establecer relaciones causales bajo condiciones de laboratorio estrictamente controladas. Se manipularán de forma deliberada variables independientes arquitectónicas para medir su impacto sobre variables dependientes clave de rendimiento, siendo la tasa de precisión en la evaluación de ARC-AGI y el coste computacional asociado.

3.2. Metodología de Desarrollo y Gestión del Proyecto

Debido a la naturaleza incremental de la síntesis de programas y la necesidad de adaptarse a los resultados de las pruebas continuas, el desarrollo del componente software se rige por metodologías ágiles, combinando prácticas de Scrum y Kanban. El trabajo se ha estructurado en torno a una épica principal desglosada en cuatro grandes hitos o bloques de tareas o *milestones*:

- **Milestone 1:** Investigación y análisis.
- **Milestone 2:** Escritura y documentación del trabajo.
- **Milestone 3:** Diseño y desarrollo del modelo.
- **Milestone 4:** Validación y experimentación del modelo.

El ciclo de desarrollo se ejecuta en iteraciones cortas o *Sprints* de 14 días de duración. Esta cadencia temporal permite realizar inspecciones regulares del código, facilitando la

adaptación temprana de hiper-parámetros o la corrección de derivas en el entrenamiento. El control de versiones, el seguimiento de tareas y la representación visual de los tableros Kanban se centralizan mediante un repositorio de código alojado en GitHub.

3.3. Ciclo de Vida del Modelo basado en MLOps

Alineado con las prácticas de desarrollo ágil para proyectos de Inteligencia Artificial, se ha adoptado una versión adaptada del ciclo de vida MLOps orientada a la experimentación en entornos restringidos. Este ciclo se compone de las siguientes fases iterativas:

- **Preparación de datos:** Tratamiento del corpus ARC-AGI, definiendo las rutinas de visualización y codificación espacial necesarias.
- **Desarrollo del modelo y selección algorítmica:** Diseño de las arquitecturas compactas y las rutinas de inferencia iterativa.
- **Entrenamiento y ajuste:** Ejecución de pruebas de Test-Time Training y refinamiento de hiper-parámetros bajo las restricciones de hardware.
- **Evaluación:** Medición cuantitativa del rendimiento del modelo utilizando conjuntos de evaluación de ARC-AGI-1.

Esta estructura garantiza que tanto el marco teórico como el desarrollo de la ingeniería se ejecuten de manera rigurosa, permitiendo que las conclusiones extraídas al final del proyecto sean sólidas y reproducibles.

4. Infraestructura y herramientas

4.1. Infraestructura de computación

Para el desarrollo y experimentación de este proyecto, se ha utilizado una estación de trabajo local equipada con hardware de alto rendimiento. Las especificaciones técnicas del equipo base son las siguientes:

- Sistema Operativo y Entorno: El sistema opera bajo la distribución de Linux Ubuntu 24.04.4 LTS con el kernel 6.14.0-34-generic. Para la aceleración de cargas de trabajo de IA en la GPU, se emplea la plataforma abierta de AMD, ROCm en su versión 7.2.3.
- Procesador (CPU): Intel Core i9-12900K de 12^a generación.
- Memoria Principal (RAM): El sistema dispone de aproximadamente 96 GB de memoria RAM disponible.

El componente central para el entrenamiento, este proyecto es la GPU AMD Radeon RX 9070 XT. Esta GPU de arquitectura avanzada está equipada con 64 Unidades de Cómputo, 4096 Stream Processors y cuenta con 128 aceleradores de IA dedicados específicamente a optimizar las cargas de trabajo de aprendizaje profundo. Su consumo típico (Typical Board Power) es de 304 W.

Para las tareas de Inteligencia Artificial, las dos métricas más críticas son la memoria de vídeo (VRAM) y el rendimiento en diferentes precisiones matemáticas. La tarjeta cuenta con 16 GB de memoria GDDR6 operando sobre una interfaz de 256 bits. Esta memoria proporciona un ancho de banda de hasta 640 GB/s.

La GPU ofrece un rendimiento de hasta 48.7 TFLOPs en precisión de punto flotante de 32 bits (FP32) para operaciones vectoriales, y hasta 195.0 TFLOPs en precisión de punto flotante de 16 bits (FP16) para operaciones matriciales. En la siguiente tabla 4.1 se muestra el rendimiento de la GPU en diferentes niveles de precisión y formatos, incluyendo el impacto de la dispersión estructurada en el rendimiento.

Nivel de Precisión	Formato	Rendimiento Base	Rendimiento con Dispersión Estructurada
Precisión Simple (Vector)	FP32	48.7 TFLOPs	-
Media Precisión (Matriz)	FP16	195.0 TFLOPs	389.0 TFLOPs
Precisión de 8 bits (Matriz)	FP8 / INT8	389.0 TFLOPs / TOPs	779.0 TFLOPs / TOPs
Precisión de 4 bits (Matriz)	INT4	779.0 TOPs	1557.0 TOPs

Tabla 4.1: Rendimiento por nivel de precisión y formato

4.2. Entorno de desarrollo

Se ha un amplio ecosistema de paquetes y librerías para el desarrollo y la experimentación. En la tabla 4.2 se muestra una lista de las principales librerías utilizadas, junto con sus versiones y abreviaciones para su referencia en el resto del documento.

Nombre	Versión	Abreviación
PyTorch	2.12.0+rocm7.2	torch
Torchaudio	2.11.0+rocm7.2	torchaudio
Torchvision	0.27.0+rocm7.2	torchvision
Huggingface_hub	0.34.4	huggingface_hub
Packaging	24.1	packaging
Ninja	1.13.0	ninja
Wheel	0.45.1	wheel
Setuptools	80.9.0	setuptools
Setuptools-scm	9.2.0	setuptools-scm
Pydantic-core	2.33.2	pydantic-core
Triton	3.3.0	triton
Matplotlib	3.10.0	matplotlib
Numpy	2.2.2	numpy
Jupyter-events	0.12.0	jupyter-events
Jupyter-lsp	2.2.5	jupyter-lsp
Jupyter_client	8.6.3	jupyter_client
Jupyter_core	5.7.2	jupyter_core
Jupyter_server	2.16.0	jupyter_server
Jupyter_server_terminals	0.5.3	jupyter_server_terminals
Jupyterlab	4.4.2	jupyterlab
Jupyterlab_pygments	0.3.0	jupyterlab_pygments
Jupyterlab_server	2.27.3	jupyterlab_server
Keras	3.9.2	keras
Notebook	7.4.2	notebook
Tensorboard	2.19.0	tensorboard
Tensorboard-data-server	0.7.2	tensorboard-data-server
Tensorflow	2.19.1	tensorflow_rocm

Tabla 4.2: Librerías de entorno en Python, versión y abreviación

4.3. Utilidades externas de ARC-AGI

5. Desarrollo

Pendiente de redactar.

5.0.1. Técnicas de aumento de datos

Pendiente de redactar.

5.1. Arquitectura y entrenamiento

Pendiente de redactar.

6. Resultados

Pendiente de redactar.

7. Planificación

Pendiente de redactar.

8. Estudio económico

Pendiente de redactar.

9. Conclusiones y Trabajo Futuro

Pendiente de redactar.

Referencias

- Aristotle, y Boeri, M. D. (2010). *Acerca del alma*. Colihue Buenos Aires.
- Chollet, F. (2019a). *GitHub - fchollet/ARC-AGI: The Abstraction and Reasoning Corpus* — *github.com*. <https://github.com/fchollet/ARC-AGI>. ([Accessed 20-05-2026])
- Chollet, F. (2019b). On the measure of intelligence. *arXiv preprint arXiv:1911.01547*.
- González, R. (2011). Descartes: las intuiciones modales y la inteligencia artificial clásica. *Alpha (Osorno)*(32), 181–198.
- Hernández-Orallo, J. (2017). Evaluation in artificial intelligence: from task-oriented to ability-oriented measurement. *Artificial Intelligence Review*, 48(3), 397–447.
- Hurbans, R. (2020). *Grokking artificial intelligence algorithms: Understand and apply the core algorithms of deep learning and artificial intelligence in this friendly illustrated guide including exercises and examples*. Manning.
- Kahneman, D. (2011). Thinking, fast and slow. *Farrar, Straus and Giroux*.
- Legg, S., Hutter, M., y cols. (2007). A collection of definitions of intelligence. *Frontiers in Artificial Intelligence and applications*, 157, 17.
- Minsky, M. (1968). *Matter, mind, and models in: Ml minsky (ed.): Semantic information processing*. MIT Press.
- Newell, A., Shaw, J. C., y Simon, H. A. (1962). The processes of creative thinking. En *Contemporary approaches to creative thinking, 1958, university of colorado, co, us; this paper was presented at the aforementioned symposium*.
- Perez, D., Samothrakis, S., y Lucas, S. (2014). Knowledge-based fast evolutionary mcts for general video game playing. En *2014 ieee conference on computational intelligence and games* (pp. 1–8).
- Spelke, E. S., y Kinzler, K. D. (2007). Core knowledge. *Developmental science*, 10(1), 89–96.
- Świechowski, M., Godlewski, K., Sawicki, B., y Mańdziuk, J. (2023). Monte carlo tree search: A review of recent modifications and applications. *Artificial Intelligence*

Review, 56(3), 2497–2562.

Waxman, W. (2014). *Kant's anatomy of the intelligent mind*. OUP USA.

Wind, J. S. (2020). Dsl solution to the arc challenge. URL <https://github.com/top-quarks/ARC-solution>.

A. Apéndice